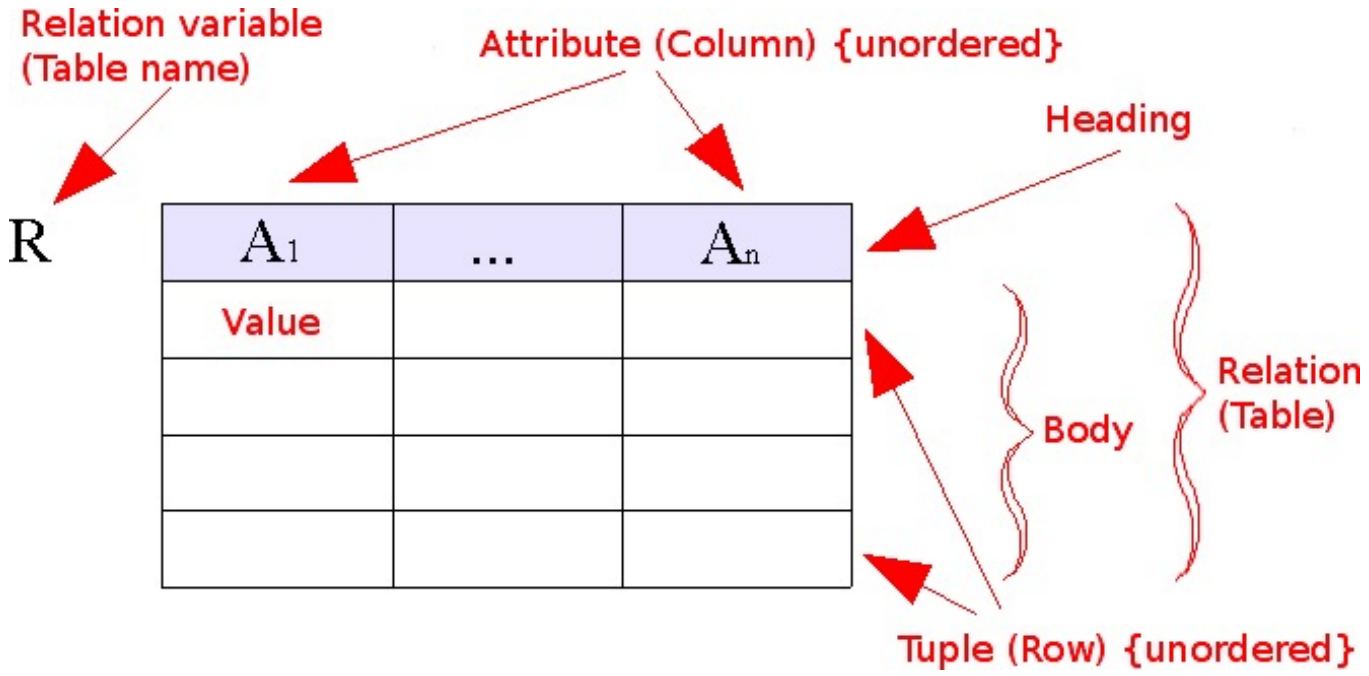
CÁTEDRA ADMINISTRACIÓN DE BASES DE DATOS
UNIVERSIDAD TECNOLÓGICA NACIONAL – FACULTAD REGIONAL LA PLATA
- INGENIERÍA EN SISTEMAS DE INFORMACIÓN –

Bases de datos relacionales

SQL avanzado

La relación base R (tabla)

- Contiene **claves**. Cada clave es un atributo (columna) cuyo valor es único en cada tupla. Existen distintos tipos.
- Las claves se **utilizan para ordenar y/o identificar filas en una tabla**.
- Las claves a menudo se indican subrayando el conjunto de atributos clave.



Tipos de claves

- **Clave candidata**: es una columna, o conjunto de columnas, en una tabla que puede identificar de forma única cualquier registro de base de datos sin hacer referencia a ningún otro dato. Cada tabla puede tener una o más claves candidatas, pero una sola será única: la clave primaria.

- **Clave primaria (PK)**: la clave primaria o principal de una relación identifica de forma única una tupla dentro de la relación (tabla). Debe ser **única para toda la relación y no puede ser nula**. Además cada relación solo puede tener una clave principal. Deberemos seleccionar una clave candidata como clave primaria.
  - **Clave compuesta**: es una clave primaria compuesta por más de una columna.

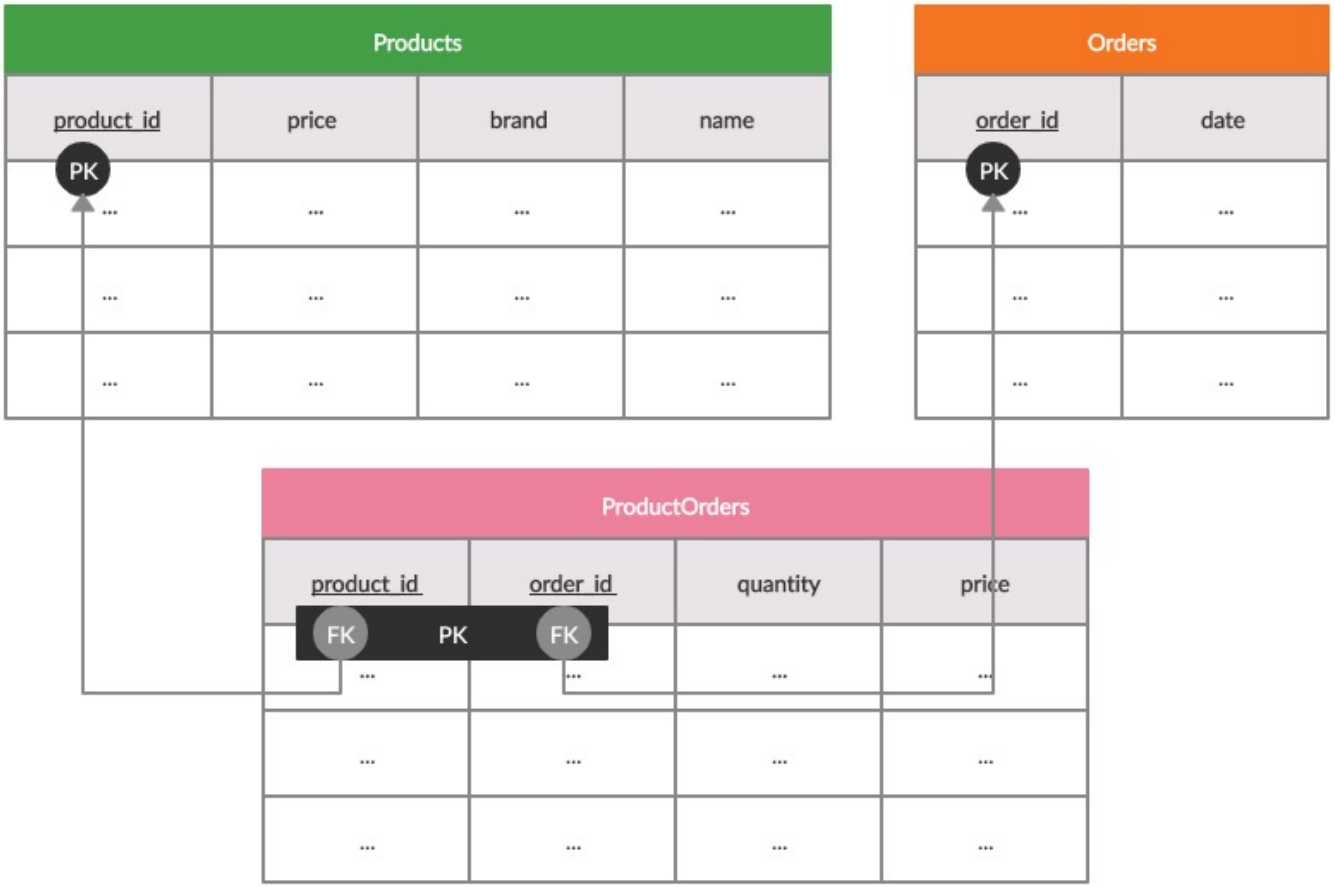
  Algunos DBMS crean automáticamente una clave primaria interna si no se define una. La generación automática de claves primarias enteras únicas **en la mayoría de los casos serán valores incrementales automáticos**. *¿esto es bueno?* Depende.

Tipos de claves (cont.)

- **Clave foránea (FK)**: referencia a una clave de otra tabla. No tienen porque ser claves únicas en la tabla donde están, pero sí donde están referenciadas: **Integridad de la información**. Actúa como una referencia cruzada entre tablas, estableciendo así un vínculo entre ellas.

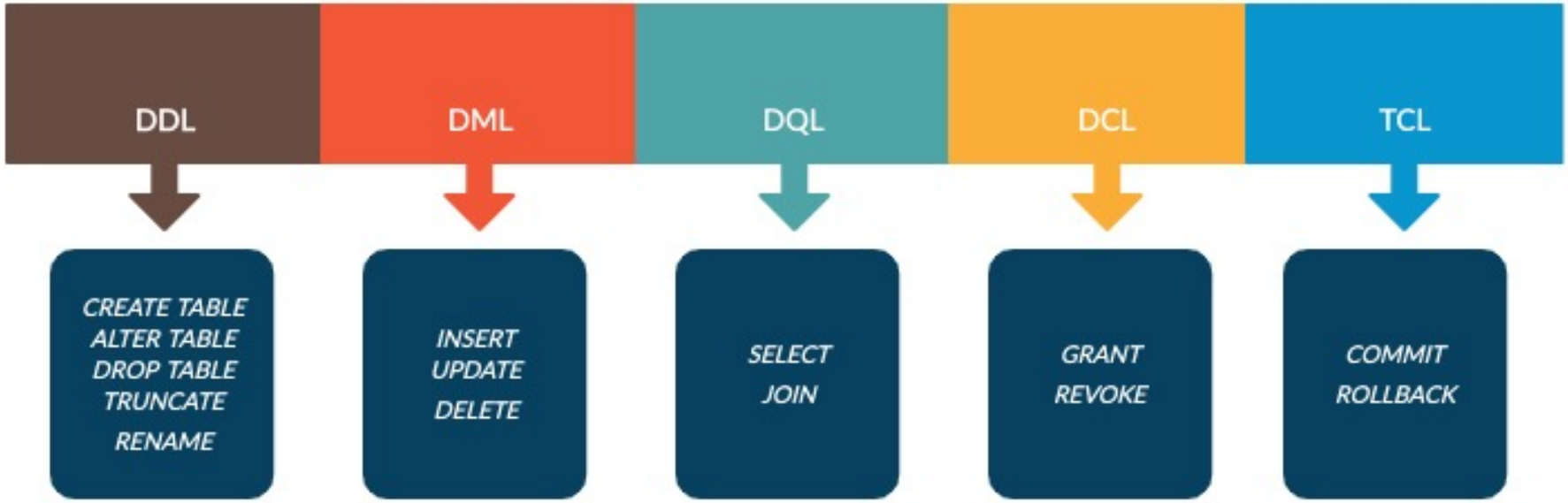
- **Clave de índice**: un índice de base de datos es una estructura de datos que ***"mejora"*** la velocidad de las operaciones de **recuperación de datos** en una base de datos, sin tener que buscar en todas las tuplas cada vez que se quiera acceder a datos de una tabla.
  - Los índices **se pueden crear utilizando una o más columnas** de una tabla, y buscarán proporcionar la base para búsquedas aleatorias rápidas y para un acceso eficiente a los registros ordenados.

Tipos de claves (cont.)

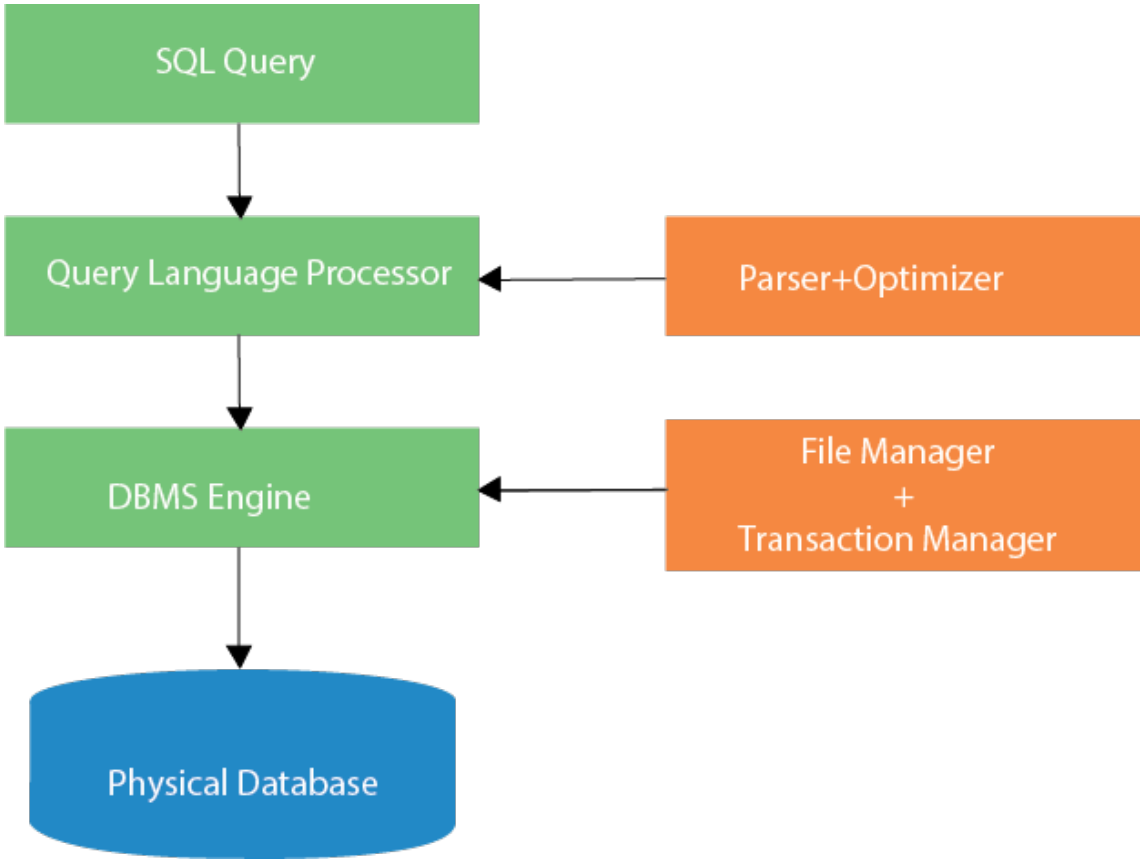


Sub-lenguajes SQL

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Query Language (DQL)
- Data Control Language (DCL)
- Transaction Control Language (TCL)



Proceso SQL


SQL Query
Query Language Processor
Parser+Optimizer
DBMS Engine
File Manager
+
Transaction Manager
Physical Database

Restricciones
y
Reglas de integridad

Restricciones y Reglas de integridad

Muchas veces los datos almacenados pueden tener inconsistencias. Para reducir dichas inconsistencias se utilizan restricciones a la hora de cargar nuevos registros en la base de datos.

Restricciones y Reglas de integridad

NOT NULL − Si se quiere que una columna no pueda contener valores nulos es necesario setear esta restricción.

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255) NOT NULL,
        …
);
```

SQL DDL (*Data Definition Language*)

Modificando columnas ya creadas

```
ALTER TABLE mi_tabla
        MODIFY mi_columna
                VARCHAR(50) NOT NULL;
```

```
ALTER TABLE mi_tabla
        MODIFY mi_columna
                VARCHAR(50);
```

Restricciones y Reglas de integridad

DEFAULT − proporciona un valor predeterminado a una columna cuando la instrucción INSERT INTO no proporciona un valor específico.

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255) DEFAULT 'sin nombre',
        …
);
```

Restricciones y Reglas de integridad

También se puede eliminar una restricción **DEFAULT**:

```
ALTER TABLE mi_tabla
        ALTER COLUMN mi_columna DROP DEFAULT;
```

Restricciones y Reglas de integridad

UNIQUE − evita que dos registros de la tabla tengan el mismo valor.

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255) NOT NULL  UNIQUE,
        …
);

ALTER TABLE mi_tabla
        MODIFY nombre  VARCHAR(255) NOT NULL UNIQUE
```

Restricciones y Reglas de integridad

También se puede restringir las combinaciones de valores de varias columnas:

```
ALTER TABLE mi_tabla
        ADD CONSTRAINT const_1 UNIQUE(col1, col2)

ALTER TABLE mi_tabla
        DROP CONSTRAINT const_1
```

Restricciones y Reglas de integridad

PRIMARY KEY − evita que dos registros de la tabla tengan el mismo valor.

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255) NOT NULL  UNIQUE,
        …
        PRIMARY KEY (id)
);

ALTER TABLE mi_tabla
        ADD PRIMARY KEY id
```

Restricciones y Reglas de integridad

Para PRIMARY KEY compuesta:

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255) NOT NULL  UNIQUE,
        …
        PRIMARY KEY (id, nombre)
);
```

Restricciones y Reglas de integridad

Para eliminar una PRIMARY KEY:

```
ALTER TABLE mi_tabla
        DROP PRIMARY KEY
```

SQL DDL (*Data Definition Language*)

Restricciones y Reglas de integridad

FOREIGN KEY − para enlazar dos tablas.

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255),
        …
        otro_id INT REFERENCES otra_tabla(id)
);

ALTER TABLE mi_tabla
        ADD FOREIGN KEY otro_id
        REFERENCES otra_tabla(id)
```

Restricciones y Reglas de integridad

Para eliminar una PRIMARY KEY:

```
ALTER TABLE mi_tabla
        DROP FOREIGN KEY
```

Restricciones y Reglas de integridad

CHECK − permite que una condición verifique el valor ingresado en un registro.

```
CREATE TABLE mi_tabla (
        id INT,
        nombre VARCHAR(255),
        edad INT CHECK (edad >= 18)
        …
);

ALTER TABLE mi_tabla
        ADD CONSTRAINT es_adulto CHECK(edad >= 18)
```

SQL DDL (*Data Definition Language*)

Restricciones y Reglas de integridad

```
ALTER TABLE my_table
        ADD CONSTRAINT es_adulto
        CHECK(edad >= 18 AND <= 100)

ALTER TABLE my_table
        ADD CONSTRAINT pais_superpoblado
        CHECK(pais IN ('USA','UK','India'))

ALTER TABLE my_table
        ADD CONSTRAINT es_fecha
        CHECK(fecha LIKE '--/--/----')
```

Funciones definidas por el usuario

Para restricciones o reglas de integridad más complejas, o para encapsular cierta lógica de negocio y poder reutilizarla fácilmente, se pueden utilizar funciones definidas por el usuario.

Vistas

Vistas *(Stored query)*

Una vista es una **tabla virtual** generada a partir de la ejecución de **varias consultas** sobre una o más tablas.

- **Privacidad de la información**: Mostramos a los usuarios la información que creamos conveniente. De esta manera no se tiene acceso a la tabla original con todas sus filas y columnas.
- **Optimización del rendimiento de la base de datos**: Podemos crear de consultas sobre vistas complejas, es decir, vistas cuya información ha sido extraída y creada a través de SELECT complejas.
- **Tablas de prueba**: Para tener la seguridad de que los datos reales no serán modificados, especialmente en ambientes de desarrollo. Esto solo es posible definiendo los permisos necesarios (GRANT).
- **Ahorro de almacenamiento**: las vistas guardan la “consulta”, no la “tabla” con sus datos.

Vistas *(Stored query)* pero…

Cuidado que no todo es bueno:

- **Cambios en la tabla original**: Si una columna es eliminada, por ejemplo, la vista dejará de funcionar. Para *reporting* no es algo de lo que debamos preocuparnos.
- **Performance**: El usuario final puede no saber qué hace internamente la vista, y puede hacer uso de muchos recursos y tiempo.
- Parecidas pero no lo mismo que una **MATERIALIZED VIEW** de SQL.

Vistas *materializadas*

Las vistas materializadas fueron implementadas por primera vez en el gestor de base de datos Oracle (versión 8i).

- MySQL/MariaDB no tiene soporte nativo para vistas materializadas, sin embargo se pueden implementar soluciones aproximadas usando permisos, triggers, stored procedures.
- PostgreSQL tiene soporte native.
- En IBM DB2 se les llama "materialized query tables".
- Microsoft SQL Server tiene una característica similar llamada "vistas indexadas”.

¿Stored procedure o Vistas?

Una **vista** es una *representación virtual* de una tabla. Se puede utilizar como una tabla, hacer JOINs, etc. Se utilizan para **leer datos y hacer reporting**. Las vistas serán generalmente más rápidas, ya que no estarán modificando datos “reales”. No son parametrizables.

Un **stored procedure** es una **función normalmente utilizado para agregar nuevos datos, actualizarlos o eliminarlos**. También utilizados para devolver datos individuales/agrupados. Son parametrizables.

SQL DDL (*Data Definition Language*)

Creación de vistas – CREATE VIEW statement

Sentencia empleada para la **creación de una vista**. Una vista es una tabla virtual que se crea como el resultado de una SELECT.

CREATE VIEW <view_name> **AS**
SELECT <column_list>
FROM <table_list>
[**WHERE** <conditions>];

- **<view_name>:** nombre de la vista.

Creación de vistas – CREATE VIEW statement

```
SELECT * FROM InfoAccidente
WHERE id_tipo_via = 'A';
```

```
CREATE VIEW accidentes_via_a AS
SELECT * FROM InfoAccidente
WHERE id_tipo_via = 'A';

SELECT * FROM accidentes_via_a;
```

SQL DDL (*Data Definition Language*)

Modificación de vistas – CREATE OR REPLACE VIEW statement

Sentencia empleada para la **modificación de una vista**.

```
CREATE OR REPLACE VIEW <view_name> AS
SELECT <column_list>
FROM <table_list>
[WHERE <conditions>];
```

SQL DDL (*Data Definition Language*)

Eliminación de vistas – DROP VIEW statement

Sentencia empleada para la **eliminación de una vista** de la base de datos.

DROP VIEW <view_name>;

¡Atención! Al borrar una vista no se produce borrado de registros de base de datos.

Creación de usuarios y privilegios

Creación de usuarios



Los usuarios de una base de datos determinan quién puede tener acceso a la misma. También se puede establecer a que objetos de la base de datos y que tipo de permisos sobre los objetos tienen los usuarios (lectura, escritura, lectura/escritura…).

Para ver los usuarios que actualmente tienen acceso a nuestras bases de datos:

```
SELECT * FROM mysql.user;

SELECT * FROM mysql.user
WHERE User != 'root' AND User NOT LIKE 'mysql.%';

SELECT CURRENT_USER();
(para usuarios anónimos SELECT USER();)
```

SQL avanzado

Creación de usuarios y privilegios



CREATE USER '<user>'@'<server>' **IDENTIFIED BY** '<pass>';

- **<user>:** nombre del usuario a crear.
- **<server>:** base de datos donde se creará el usuario.
- **<pass>:** contraseña.

mysql -u <username> **-p**

CREATE USER 'user_2'@'%.%.%.%' IDENTIFIED BY 'user_2'; -- comodín para cualquier IP

https://dev.mysql.com/doc/refman/5.7/en/create-user.html

Creación de usuarios (cont.)




```
RENAME USER 'test'@'localhost' TO 'otro'@'localhost';
RENAME USER 'test'@'localhost' TO 'test'@'otro_host';

DROP USER 'test'@'localhost';
DROP USER IF EXISTS 'test'@'localhost';

ALTER USER 'test'@'localhost'
IDENTIFIED BY 'nueva-contraseña';

SET PASSWORD ['test'@'localhost'] = 'nueva-contraseña';
```

SQL avanzado

Privilegios y permisos



GRANT <priv_type> **ON** <object> **TO** '<user>'@'<server>';
REVOKE <priv_type> **ON** <object> **FROM** '<user>'@'<server>';

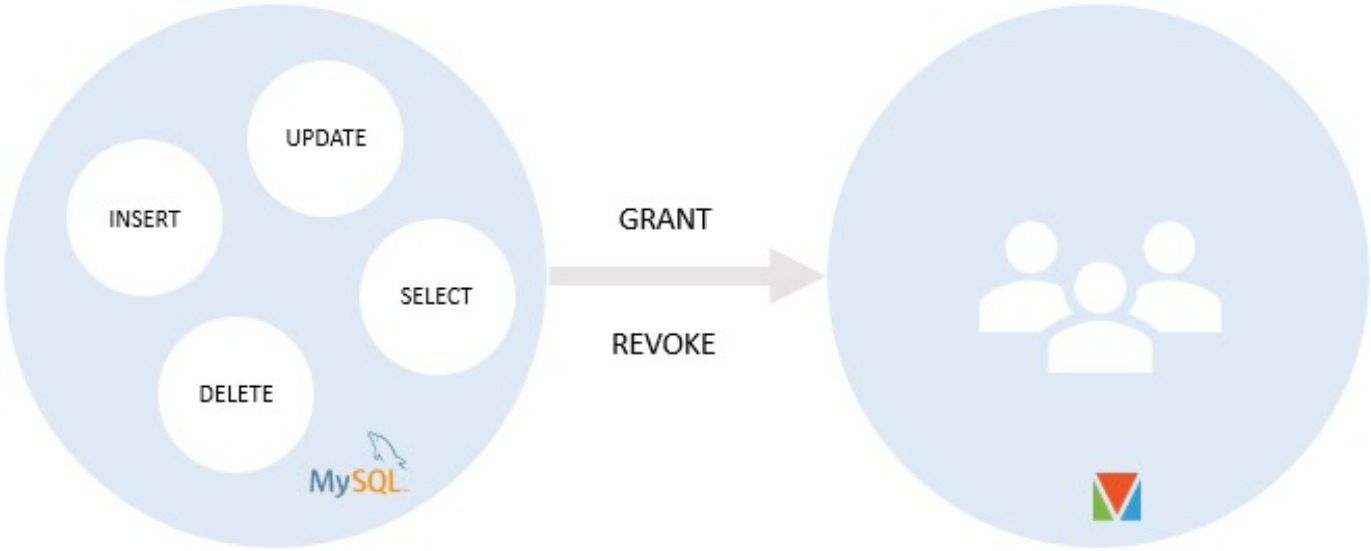
[**GRANT** | **REVOKE**] **EXECUTE**
ON [**PROCEDURE** | **FUNCTION**] <object>
[**TO** | **FROM**] '<user>'@'<server>';

- **<user>:** nombre del usuario a crear.
- **<server>:** host/servidor donde está el usuario.
- **<priv_type>:** tipo de privilegio que se dará al usuario.
- **<object>:** objeto u objetos para los que se dará privilegios al usuario.

https://dev.mysql.com/doc/refman/5.7/en/grant.html

SQL avanzado

Privilegios y permisos



Tipo de privilegios



- **SELECT:** posibilidad de leer datos de las tablas;
- **INSERT**: posibilidad de insertar nuevos datos en las tablas;
- **UPDATE**: posibilidad de actualizar datos de las tablas;
- **DELETE**: posibilidad de eliminar datos de las tablas;

- **INDEX**: posibilidad de crear índices en tablas existentes;
- **CREATE**: posibilidad de crear nuevas tablas;
- **ALTER**: posibilidad de modificar estructura y definición de tablas;
- **DROP**: posibilidad de eliminar tablas;

- **GRANT OPTION**: posibilidad de dar privilegios a otros usuarios;
- **ALL**: todos los permisos anteriores, menos **GRANT OPTION**.

Ejemplos

```
// todos los permisos en todas las tablas de la BD accidentes
GRANT ALL ON accidentes.* TO 'user_1'@'localhost';

GRANT SELECT, INSERT, UPDATE, DELETE
ON mi_bd.mi_tabla (col_1, col_2)
TO 'user_2'@'localhost'

// a todos los usuarios, con conexiones locales
GRANT SELECT ON mi_bd.mi_tabla TO '*'@'localhost'

// al usuario 'test', con cualquier tipo de conexión.
GRANT SELECT ON mi_bd.mi_tabla TO 'test'@'%'
```

Ejemplos

```
// ver permisos de usuario
SHOW GRANTS FOR 'user_1'@'localhost';

// eliminar permisos de SELECT en vista
REVOKE SELECT ON mi_bd.mi_vista FROM '*'@'localhost';

SHOW GRANTS FOR 'user_1'@'localhost';
```

- El privilegio **USAGE** significa "no tiene privilegios".
- Los privilegios se asignan ***por objeto***.

Privilegios avanzados

La sentencia ALTER USER se puede utilizar también para modificar opciones en cuanto a recursos, contraseña y bloqueo:

ALTER USER 'user_1'@'localhost' **ACCOUNT** [**LOCK** | **UNLOCK**];

ALTER USER 'user_1'@'localhost' **WITH**
[**MAX_QUERIES_PER_HOUR** | **MAX_UPDATES_PER_HOUR**];

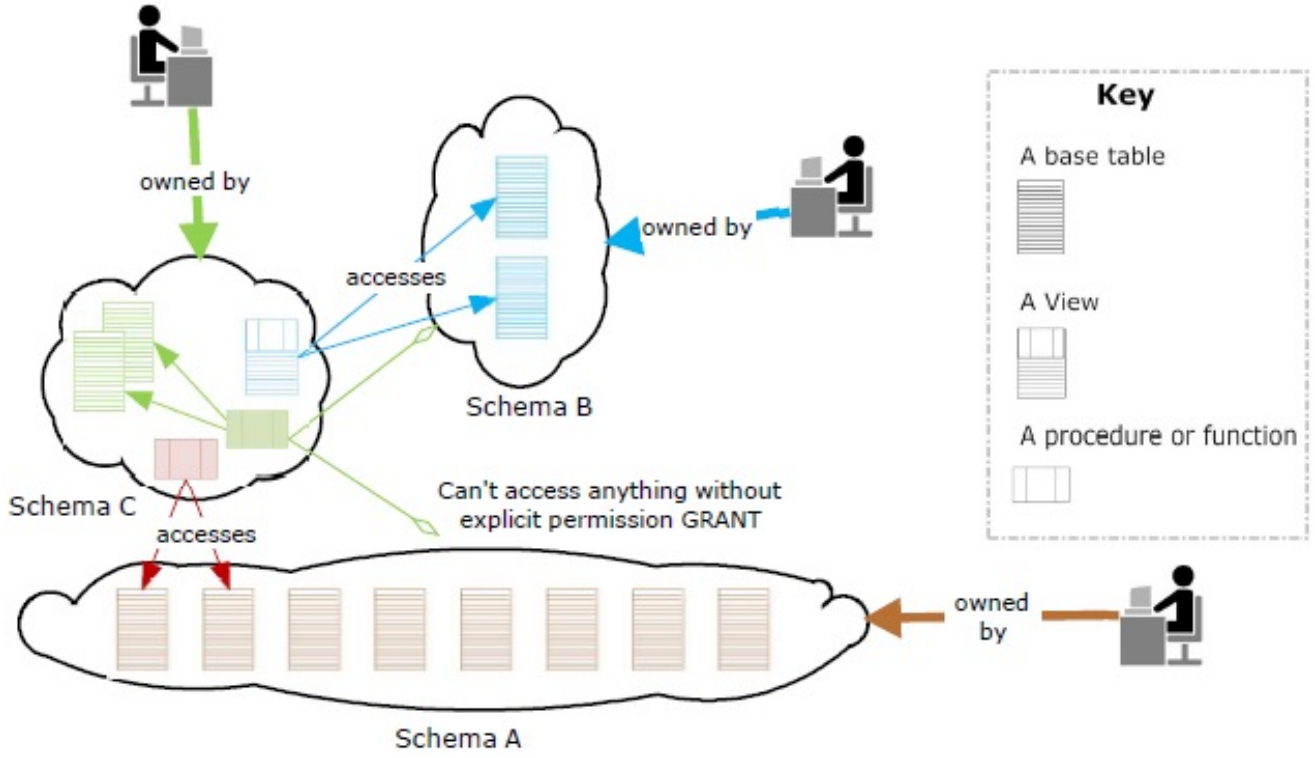
ALTER USER 'user_1'@'localhost' **PASSWORD EXPIRE NEVER**;
ALTER USER 'user_1'@'localhost' **PASSWORD EXPIRE INTERVAL 60 DAY**;

Gestión de roles

Gestión de roles

Cuando el conjunto de permisos se repiten en distintas personas, es conveniente utilizar **roles**.

- Desarrollo
- Marketing
- Gerencia
- etc.



Creación de roles

```
// al igual que los usuarios, los roles tendrán 'rol'@'host'  (default %)
CREATE ROLE <nombre_rol>;

// asignar permisos al rol
GRANT ALL ON accidentes.* TO <nombre_rol>;

// quitar permisos del rol
REVOKE SELECT ON mi_bd.mi_vista FROM <nombre_rol>;

// ver permisos de un rol
SHOW GRANTS FOR <nombre_rol>;

// eliminar un rol
DROP ROLE <nombre_rol>;
```

Asignación de roles

// asignar rol a usuario
GRANT <nombre_rol> **TO** 'user_1'@'localhost';

// quitar rol a usuario
REVOKE <nombre_rol> **FROM** 'user_1'@'localhost';

// copiar privilegios de usuario A a usuario B
GRANT <usuario_A> **TO** <usuario_B>;

Activando roles

Ahora si nos conectamos con el usuario que tiene un rol asignado nos dará error.

Debemos definir qué roles queremos que estén activos cuando el usuario se conecte.

```
// desde root: definir roles usuario
SET DEFAULT ROLE [ <nombre_rol> | ALL | NONE ]
TO 'user_1'@'localhost';

// ver permisos y roles del usuario
SHOW GRANTS FOR 'user_1'@'localhost';

// ver permisos y roles del usuario, con detalle del rol
SHOW GRANTS FOR 'user_1'@'localhost' USING <nombre_rol>;

// desde usuario: ver qué roles tengo asociados a mi usuario
SELECT CURRENT_ROLE();
```

Transacciones

Transacciones (I)

Mediante estas sentencias SQL podemos **crear una transacción** (operación ACID) sobre la información de la base de datos. Al finalizar la modificación de información, podemos confirmar los cambios o descartarlos.

```
START TRANSACTION;
<insert|update|delete, sentencias…>;
COMMIT|ROLLBACK;
```

- **START TRANSACTION:** comienza la transacción en base de datos.
- **COMMIT:** finaliza la transacción confirmando los cambios.
- **ROLLBACK:** finaliza la transacción descartando los cambios.



Transacciones (II)

Por defecto, MySQL se ejecuta con el **modo autocommit habilitado**. Esto quiere decir, que tan pronto se ejecuta una actualización de información en base de datos, los cambios se convierten en permanentes. No se puede dar marcha atrás a los cambios. Se puede cambiar este modo de funcionamiento con la siguiente sentencia.

SET autocommit = 0|1;



Ejemplo transacción

```
START TRANSACTION;
INSERT INTO playlist(id, title, is_public)
VALUES(2, 'Mis canciones favoritas (segunda parte)', 1);

INSERT INTO user_has_playlist(id_user, id_playlist)
VALUES(1, 2);

INSERT INTO playlist_has_song(id_playlist, id_song)
VALUES(2, 3);

INSERT INTO playlist_has_song(id_playlist, id_song)
VALUES(2, 4);
COMMIT;
```

Ejemplo transacción

```
START TRANSACTION;
INSERT INTO playlist(id, title, is_public)
VALUES(3, 'Mis canciones favoritas (tercera parte)', 1);

INSERT INTO playlist(id, title, is_public)
VALUES(3, 'Mis canciones favoritas (tercera parte)', 1);

#error?
ROLLBACK;
```

Triggers

Triggers

- El **trigger o disparador** MySQL es un objeto de la base de datos que está asociado con tablas.

- Son comunmente utilizados para definir **reglas de negocio**.

- El trigger se puede ejecutar cuando se ejecuta una de las siguientes sentencias MySQL en una tabla: INSERT, UPDATE y DELETE. **Se puede invocar antes o después del evento.**

- Una buena práctica es hacer que los triggers ejecuten diréctamente **stored procedures**, para poder replicar su funcionamiento luego.

Triggers

Las palabras clave **OLD** y **NEW** nos dan acceso a las columnas de las filas afectadas por el trigger.

```
CREATE TRIGGER history [BEFORE| AFTER] [INSERT|DEL.|UPD.]
ON mi_tabla

FOR EACH ROW
BEGIN
        INSERT INTO tabla_logs(id, accion, fecha)
        VALUES(NEW.id, 'INSERT', CURDATE());
END;
```

Delimiter

```
DELIMITER //
CREATE TRIGGER history AFTER INSERT
ON my_table

FOR EACH ROW
BEGIN
        INSERT INTO tabla_logs(id, accion, fecha)
        VALUES(NEW.id, 'INSERT', CURDATE());
END//
```

Se utilizan **DELIMITADORES** para poder utilizar **;** como parte del trigger.

Backups/Restore

mysqldump [opciones] > *backup.sql*

mysqldump –u root -p **mi_bd** t1 t2 t3 > backup_t_tablas.sql
mysqldump –u root -p **mi_bd** > backup_todas_tablas.sql

mysqldump -u root -p **--databases bd1 bd2** > backup_2_bd.sql
mysqldump -u root -p **--all-databases** > todas_dbs.sql

mysqldump -u root -p **mysql** user > backup_users.sql

restaurar base de datos
mysql mi_bd < backup.sql
(también está el comando **mysqlimport**)

https://dev.mysql.com/doc/refman/5.6/en/mysqldump.html

Materiales

- Try SQL: https://www.codeschool.com/courses/try-sql
- SQL Tutorial: http://www.w3schools.com/sql/

- SQL and Relational Theory
- SQL Pocket Guide
- Learning SQL
- MySQL Cookbook

